

그래프 엔지니어링

에이전트의 갈림길을 누가 소유하는가

온보딩 노트 · 2026-08-19 · 읽는 데 20분, 실습 10분

다 읽으면 할 수 있는 것 — 임의의 에이전트 시스템을 앞에 놓고 **갈림길 하나를 집어 주인을 판정**하고, 그 주인을 모델에서 코드로 옮길지 결정한다.

1. 왜 지금인가 — 수십 년 된 이론이 재발견된 이유
2. 부품 4개 — 상태 · 노드 · 엣지 · 체크포인트
3. 판정 — 값인가 경로인가
4. 실전 — 미니 회의록 에이전트
5. 쓰지 말아야 할 때
6. 판단 기준 체크리스트

1. 왜 지금인가

그래프·상태 머신은 새 이론이 아니다. 항공권 예약, 결제 승인, CI 파이프라인 — 소프트웨어는 수십 년 전부터 "노드 + 엣지 + 상태"로 돌아갔다. 그런데 그 수십 년 동안 아무도 이걸 "그래프 엔지니어링"이라 부르지 않았다.

상태 머신은 그래프에 영속 상태와 허용 전이 목록을 더한 것이다 — 지금 어느 칸인지를 저장하고, 목록에 없는 전이는 거절한다. 중단·재개·감사가 필요할 때 그래프를 여기까지 승격시킨다.

이유는 하나다. **엣지의 주인 후보가 코드 하나뿐이었다**. 모든 부품이 결정적이니 "다음 단계를 누가 정하나"라는 질문 자체가 성립하지 않았다. 후보가 하나면 선택이 없고, 선택이 없으면 엔지니어링이 아니라 그냥 프로그래밍이다.

2023년, 소프트웨어 역사상 처음으로 **스스로 다음 행동을 정할 수 있는 부품(LLM)**이 등장했다. 엣지의 두 번째 후보. 업계의 첫 반응은 정반대 극단이었다 — "갈림길 전부를 모델에게 주자"(AutoGPT류 자율 에이전트). 결과는 무한 루프, 비용 폭발, 같은 입력에 매번 다른 경로, 중간에 죽으면 처음부터 다시.

그래서 그래프 엔지니어링은 신기술이 아니라 **반성문**이다 — "모델에게 전부 줬더니 안 되더라. 미리 답을 아는 갈림길만 도로 코드로 뺏어오자." Anthropic의 workflow/agent 구분(2024.12)과 Cognition의 "멀티에이전트 만들지 마라"(2025)가 그 반성의 기록이고, Temporal·Inngest 같은 수십 년 계보의 내구 실행 엔진이 갑자기 AI 에이전트 문서를 쏟아내는 이유다.

오래된 건 표지판과 대피소이고, 새로운 건 표지판을 뽑아갈 수 있는 두 번째 손이 처음 생겼다는 사실이다.

2. 부품 4개

프리미티브는 4개뿐이고, 이 4개는 루프에도 그래프에도 똑같이 있다. 어떤 에이전트 시스템을 뜯어도 나온다.

부품	하는 일	산출물
상태 state	모두가 읽고 쓰는 공유 데이터	지금까지의 진실
노드 node	상태를 받아 일하고 갱신된 상태를 돌려주는 구간	값
엣지 edge	노드가 끝난 뒤 "다음 노드는 무엇인가"에 답하는 결정	다음 노드의 이름
체크포인트	지금까지의 상태를 저장 — 방향은 정하지 않는다	재개 지점

엣지와 체크포인트를 섞지 않기 — 등산

가장 흔한 혼동이라 비유로 먼저 못 박는다.

산	시스템	하는 일
갈림길 표지판	엣지 (edge)	다음에 어디로 갈지
대피소	체크포인트 (checkpoint)	엎어지면 여기서부터 다시

대피소는 길을 안 알려주고, 표지판은 재워주지 않는다. 체크포인트는 방향과 아무 관계가 없다.

루프는 그래프의 특수 케이스다

루프(agent)와 그래프(workflow)는 다른 종류의 물건이 아니다. 같은 부품 4개를 쓰고, 엣지의 소유자 한 칸만 다르다. 루프 = 노드 1개 + 자기 자신으로 돌아오는 조건부 엣지.



그림 1. 같은 부품 4개, 달라지는 것은 엷지 소유자 한 칸

부품	그래프 (LangGraph)	루프 (Claude Code)	상태 머신 (OAuth 플로우)
state	공유 state 객체	대화 컨텍스트	DB 한 행
node	함수 여러 개	모델 호출+도구 — 사실상 1개	HTTP 핸들러 3개
edge	코드의 라우팅 함수	모델의 출력	코드의 조건부 전이
checkpoint	checkpointer	세션 resume	트랜잭션 + version 컬럼

edge 행만 다르다. 이 표가 이 문서 전체의 요약이다.

3. 판정 — 값인가 경로인가

호출 주체와 결정 주체는 다른 자리다

실행하는 자리는 언제나 프로그램이다. AI가 새로 앓은 자리는 "다음에 뭘 할지 이름을 대는 자리" 하나뿐이다.

```
step = model.decide(state) # 함수 '이름'이 여기서 정해진다 (결정)
state = T00LS[step.name](**step.args) # 함수를 '호출'하는 건 이 파이썬 줄 (실행)
```

모델은 "transcribe" 라는 문자열을 반환했을 뿐 실행 권한을 가진 게 아니다. 내비게이션이 "다음 사거리 좌회전"이라 말해도 핸들을 꺾는 건 운전자인 것과 같다 — 다만 프로그램은 운전자와 달리 내비 말을 거부하지 못한다.

노드 경계는 발견하는 게 아니라 긋는 것

노드와 엣지를 상자·화살표 같은 **사물**로 배우면 반드시 헛갈린다. 둘은 사물이 아니라 역할이다. 같은 시스템을 노드 1개로도 5개로도 그릴 수 있고 둘 다 맞다. 선을 긋는 실익은 두 가지뿐 — 그 경계에서 **체크포인트를 찍고 싶은가**, 그 경계에서 **경로가 갈리는가**. 둘 다 아니면 긋지 마라.

그래서 "이건 노드 안인가 엣지인가"는 답이 없는 질문이고, 답이 있는 질문은 하나뿐이다.

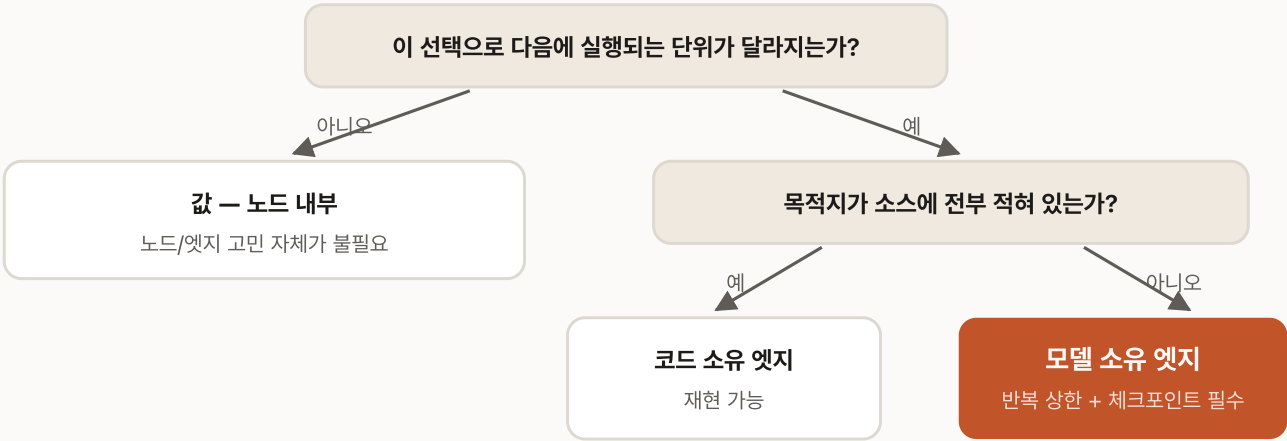


그림 2. 겹모습(if문이나 도구 호출이나)은 묻지 않는다. 목적지만 묻는다

사례	무엇을 바꾸나	정체	주인
고정된 단계 순서	—	고정 엣지	코드
실패 시 건너뛰기 (except)	경로 (목적지 열거됨)	조건 엣지	코드
LLM 출력에 "긴급" 있으면 알림	경로 (목적지 열거됨)	조건 엣지	코드
노드 안에서 모델이 도구 선택	값	노드 내부	—
LLM 출력 문자열 후처리	값	노드 내부	—
모델이 handoff_to_X 도구 선택	경로 (목적지가 응답에서 나옴)	엣지	모델
스킬 문서에 적힌 단계 순서	경로	엣지	프롬프트 권고

마지막 행이 세 번째 칸이다. 강제력에는 세 단계가 있다 — **코드가 강제 > 프롬프트가 권고 > 모델이 매번 결정**. 프롬프트에 "절대", "반드시" 같은 강조가 많다는 것 자체가 그 순서가 강제가 아니라 권고라는 증거다. 코드가 소유한 순서에는 그런 문장이 필요 없다. 애초에 못 어기니까.

4. 실전 — 미니 회의록 에이전트

여기서부터는 돌아가는 코드다. 전사본을 회의록으로 만드는 그래프이며, 의존성 없는 파이썬 한 파일(149줄, 그래프 본체는 100줄 이하)이라 바로 실행된다.

03-graph/toy/

<code>mini_minutes.py</code>	그래프 — 노드 5개, 엣지 전부를 소유한 함수 1개, 체크포인트
<code>fake_llm.py</code>	결정적 가짜 모델 (API 키 불필요)
<code>real_llm.py</code>	실제 Claude 호출 — 시그니처 동일
<code>sample.txt</code>	샘플 전사본
<code>test_mini_minutes.py</code>	자기 검사 3개

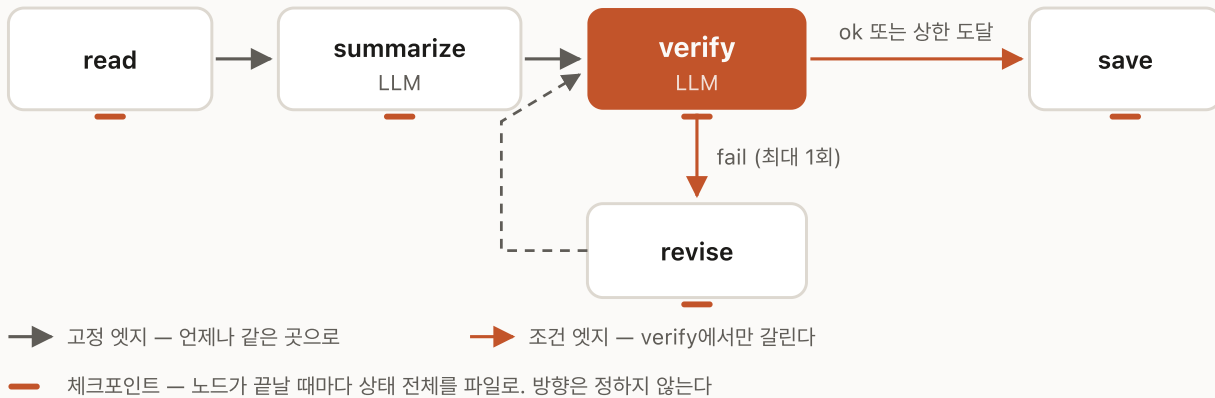


그림 3. 화살표 하나하나가 엣지다. 갈림길은 verify 뒤 한 곳뿐이고, 그림에 안 그린 시작(queued→read)·종료(saved→END) 엣지까지 전부 `next_node()` 함수 하나가 소유한다

① 엣지 — 이 함수 하나가 그래프 엔지니어링의 전부다

```
MAX_REVISIONS = 1    # 코드가 소유한 상한. 모델은 이 숫자를 바꿀 수 없다.

def next_node(state) -> str | None:
    """다음 노드의 '이름'을 돌려준다. None이면 종료."""
    stage = state["stage"]
    if stage == "queued":          return "read"
    if stage == "read":            return "summarize"
    if stage in ("summarized", "revised"): return "verify"
    if stage == "verified":
        if state["verdict"]["ok"] or state["revisions"] >= MAX_REVISIONS:
            return "save"
        return "revise"
    if stage == "saved":           return None
```

목적지 후보 — `read / summarize / verify / revise / save / None` — 가 소스에 전부 열거돼 있다. 그림 2의 판정 절차를 적용하면 **엣지의 주인은 코드**다. 모델은 `verdict["ok"]` 라는 값만 제공하고, 그 값으로 어디에 갈지는 이 함수가 정한다.

이 함수의 `if` 한 줄 한 줄이 엣지다. `read` 다음이 언제나 `summarize` 인 것도 엣지이며, 다만 갈 곳이 하나뿐이라 **고정 엣지**라 부른다. `verified` 줄만 상태를 보고 두 곳(`save / revise`)으로 갈리므로 **조건 엣지**다. 둘의 차이는 소유자가 아니라 목적지 후보의 개수뿐이고, 이 그래프에서는 전부 코드가 소유한다.

② 노드 — 값만 만들고, 방향은 절대 정하지 않는다

```
def summarize(state, llm):
    state["note"] = llm.summarize(state["transcript"])
    state["stage"] = "summarized"    # 자기가 끝났다고 기록할 뿐,
    return state                    # 다음이 어디인지는 모른다
```

노드 안에서 모델이 무슨 짓을 하든 — 도구를 열 번 부르든, 재시도를 하든 — `summarize` 다음은 언제나 `verify` 다. 그래서 그 모든 선택은 값이다.

③ 러너 — 루프 하나면 그래프가 돈다

```
while True:
    name = next_node(state)          # 엣지가 목적지를 고르고
    if name is None: return state
    state = NODES[name](state, llm)  # 노드가 일을 하고
    ckpt.write_text(json.dumps(state)) # 체크포인트가 저장한다
```

LangGraph 같은 그래프 프레임워크가 하는 일도 본질은 이 루프다. 프레임워크가 여기에 얹어주는 것은 세 가지다.

- **병렬 실행** — 위 루프는 노드를 한 번에 하나씩만 돌리지만, 프레임워크는 독립적인 노드 여러 개(예: 청크 3개를 각각 요약)를 동시에 돌려준다.
- **reducer(병합 규칙)** — 병렬로 돌아온 결과들이 **같은 상태 키**에 쓰려고 할 때 어떻게 합칠지의 규칙. "뒹어쓰다 / 이어붙인다"를 키마다 정해둔 것이다. 위 토이처럼 한 번에 하나만 돌면 충돌이 없으니 필요 없다 — 병렬을 도입하는 순간에만 태어나는 부품이다.
- **영속 백엔드** — 위 루프의 `checkpoint.json` 파일 자리에 Postgres 같은 DB를 끼워주는 것. 저장한다는 개념은 같고, 저장 장소만 튼튼해진다.

즉 프레임워크는 새로운 개념을 주는 게 아니라 **같은 루프에 편의 장치를 붙인 것**이고, 노드 3~5개짜리 시스템에는 위 루프로 충분하다. Anthropic도 프레임워크보다 "API 직접 조합" — 이 문서의 러너처럼 모델 API를 파이썬으로 직접 불러 조립하는 방식 — 을 먼저 권한다. 프레임워크의 추상화 계층이 실제로 나간 프롬프트와 돌아온 응답을 가려서, 문제가 생겼을 때 어디를 봐야 할지 어려워지기 때문이다.

④ 직접 확인 — 실험 3개

실험 1. 조건 엣지가 작동하는가

```
$ python mini_minutes.py sample.txt --reset
✓ read      → stage=read
✓ summarize → stage=summarized
✓ verify    → stage=verified
✓ revise    → stage=revised      ← 엣지가 fail을 잡아 되돌렸다
✓ verify    → stage=verified
✓ save      → stage=saved
완료: sample.minutes.md (재요약 1회)
```

첫 요약은 액션 아이템 owner를 "미지정"으로 남기고, 검증이 fail을 내고, 엡지가 `revise` 로 돌린다. Anthropic 패턴 카탈로그의 **evaluator-optimizer**가 이 6줄이다.

실험 2. 체크포인트가 일을 아끼는가

```
$ python mini_minutes.py sample.txt --crash verify --reset
✓ read      → stage=read
✓ summarize → stage=summarized
✗ verify 직전에 크래시 (재실행하면 여기서부터)

$ python mini_minutes.py sample.txt
체크포인트 발견 - stage=summarized부터 이어서 실행
✓ verify      → stage=verified      ← 전사·요약을 다시 하지 않는다
✓ revise      → ...
```

재개 로직은 0줄이다.

상태에 `stage` 가 있고 엡지가 상태에서 목적지를 계산하므로, 체크포인트를 저장하는 순간 재개가 공짜로 따라온다. 체크포인트를 넣었는데 재개가 없으면 좀비 작업만 쌓인다 — 저장과 재개는 항상 한 쌍이다.

실험 3. 상한이 무한 루프를 막는가

검증이 영원히 fail을 돌려줘도 `MAX_REVISIONS = 1` 때문에 `save` 로 빠진다. 모델은 이 숫자를 바꿀 수 없다 — 코드가 엡지를 소유한 실익이 정확히 이것이다. `test_mini_minutes.py` 가 이 시나리오를 자동으로 검사한다.

⑤ 모델을 갈아끼워도 그래프는 그대로다

```
$ python mini_minutes.py sample.txt --real --reset # 가짜 모델 → Claude
```

`real_llm.py` 는 `fake_llm.py` 와 시그니처가 같다. `--real` 을 켜도 노드 목록·엡지 함수·체크포인트는 **한 줄도 바뀌지 않는다**. 바뀌는 것은 노드 안에서 값을 만드는 방법뿐 — "노드 안 = 값, 노드 사이 = 경로"의 실물 증거다.

⑥ 이 토이에 일부러 없는 것

- 병렬(팬아웃/팬인)과 reducer — 노드가 한 번에 하나만 도니 병합 규칙이 필요 없다. reducer는 fan-out을 도입하는 순간에만 태어나는 부품이다. 긴 전사본을 청크로 쪼개 병렬 요약하는 날 처음 필요해진다.
- 사람 승인 게이트 — 체크포인트가 이미 있으니 `stage="awaiting_approval"` 을 추가하고 엣지에 한 줄 넣으면 된다. LangGraph의 `interrupt()` 가 하는 일이 이것이다.
- 멍등성 — `save` 가 파일을 덮어쓰므로 재실행이 안전하다. 메일 발송 같은 외부 부수효과를 노드에 넣는 순간 upsert 또는 발송 기록이 필요해진다. 체크포인트가 노드 경계에만 있으므로, 크래시한 노드는 처음부터 다시 실행된다는 사실을 잊으면 메일이 두 번 나간다.

5. 쓰지 말아야 할 때

멀티에이전트 — 금지의 정확한 사정거리

Cognition의 "Don't Build Multi-Agents"는 "에이전트를 여러 개 쓰지 마라"가 아니라 "서로 대화하며 협업하는 에이전트들을 만들지 마라"다. 근거는 하나 — 모든 행동에는 암묵적 결정이 실려 있고, 에이전트를 쪼개면 컨텍스트가 쪼개지고, 서로 다른 컨텍스트 위에서 내린 결정들은 합칠 때 충돌한다.

Anthropic이 같은 해에 멀티에이전트 리서치로 90.2% 향상을 보고한 것과 모순되지 않는다. 작업의 성질이 다를 뿐이다.

	읽기 (리서치, 탐색)	쓰기 (코드, 디자인, 문안)
서브에이전트 산출물	요약·발견 → 합치면 가산적	결정이 실린 결과물 → 합치면 충돌
병렬화	안전	위험
비용	채팅 대비 약 15배 토큰	—

Anthropic 시스템의 서브에이전트들도 **서로 대화하지 않는다**. 리드가 뿌리고, 각자 읽기만 하고, 결과는 리드 한 곳으로만 돌아온다. Cognition이 금지한 수평 대화가 애초에 없다. 그리고 최종 보고서 작성은 단일 에이전트가 했다.

여기서 **오케스트레이터**는 리더가 아니라 **자리다** — 쪼개서 뿌리고(팬아웃) 받아서 합치는(팬인) 노드 하나에 붙인 이름. 그 자리에 코드가 앞으면 스크립트, LLM이 앞으면 에이전트다. 누가 앞든 배선(수평 대화 금지, 중심으로만 합류)은 코드가 쥔다.

그래프가 실제로 깨지는 곳

실패 모드	증상	처방
컨텍스트 붕괴	fan-out 결과 전부를 합류 노드 하나에 부으면 종합 전에 컨텍스트 초과	계층적 fan-in — 배치별 요약 → 요약들만 종합
거짓 독립성	프롬프트는 서로를 모르는데 같은 파일·API 한도를 공유 — 숨은 엣지	자원 격리(worktree 등). 같은 자원에 쓰면 병렬이 아니라 엣지가 필요하다
조용한 노드 실패	200개 중 1개가 죽어도 보고서가 완성돼 보인다	fan-in에서 개수 검증 — 부족하면 부분 데이터로 종합하지 않는다

둘째 행(거짓 독립성)만 풀어 쓴다. 병렬로 돌린다는 것은 "이 둘 사이엔 엣지(의존성)가 없다"고 선언하는 것인데, 그 선언이 거짓인 경우다. 프롬프트만 보면 서로 무관하다 — A는 "auth 모듈 고쳐", B는 "db 모듈 고쳐". 그런데 둘이 **같은 작업 폴더**에서 돌면: A가 `git stash` 로 작업을 잠시 치우고, B가 자기가 한 줄 알고 `stash pop` 후 `reset --hard` — A의 작업이 통째로 증발한다. 실화다: Bun을 Rust로 이식할 때 파일 1,448개에 에이전트를 병렬로 뿌리자 2분 만에 이 사고가 났고, worktree 4개(같은 저장소를 독립 폴더로 복제)로 분리해 해결했다.

함정은 **프롬프트만 보서는 진짜 독립과 구분이 안 된다**는 것이다. 의존성은 프롬프트가 아니라 자원(폴더·파일·API 한도)에 숨어 있다. 그래서 처방이 둘로 갈린다 — 자원을 쪼갤 수 있으면 격리하고(각자 자기 폴더), 쪼갤 수 없으면 애초에 독립이 아니었던 것이니 병렬을 포기하고 화살표(순서)를 그린다.

그리고 앵커 — 검증 노드끼리 서로를 확인해도 전부 같은 시스템에서 나온 산출물을 돌려 보고 있으면 "모든 것이 일관되지만 아무것도 검증되지 않은" 그래프가 된다. 실제로 실행돼 통과한 테스트처럼 **그래프 밖의 ground truth**가 최소 하나는 있어야 한다.

6. 판단 기준

다음 갈림길을 만났을 때 이 순서로 묻는다.

If 단계의 개수와 순서를 코드 작성 시점에 열거할 수 있다 Then 그래프로 고정한다. If 모델이 실행 중에 다음 단계를 정해야만 한다 Then 루프를 쓰되 반복 상한을 코드로 건다.

If 노드 하나가 30초 이상 걸리거나 유료 API를 부른다 **Then** 그 노드 뒤에 체크포인트를 둔다. **If** 상태를 영속화했다 **Then** 그 상태로 시작 노드를 고르는 재개 엷지를 반드시 같이 만든다.

If "품질 게이트를 넣고 싶은데 기준을 코드로 못 적겠다" **Then** 기준이 아니라 **배선만** 코드로 적는다 — 건너뛴 수 없음 + 최대 N회 + 실패 시 되돌아갈 곳. 판정 자체는 모델에게 맡긴다.

If 서브에이전트들이 읽기만 하고 결과가 한 곳으로 합류한다 **Then** 병렬화해도 된다. **If** 산출물에 결정이 실린다(코드·디자인·문안) **Then** 쪼개지 말고 하나가 전체 컨텍스트를 들고 순차로 한다.

If "이 시스템은 agent인가 workflow인가"를 묻고 있다 **Then** 질문이 틀렸다. 갈림길 3개를 집어 각각의 주인을 표로 적는다.

표현 주의

"에이전트에 그래프 엔지니어링을 도입했다"는 어색하다. 도입하는 물건(프레임워크)처럼 들리기 때문이다. 실제 한 일은 갈림길 단위로 말한다 — **"요약 재시도 갈림길을 모델에서 코드로 옮겼다"**, **"전사 노드 뒤에 체크포인트를 넣어 재개 가능하게 했다"**. 그렇게 말하는 순간 문장이 검증 가능해진다. 파일과 줄을 지목할 수 있으니까.

상세 노트 — `03-graph/notes/graph-intuition.md` (비유·판정), `graph-primitives.md` (LangGraph 프리미티브 원문 정리), `multi-agent-pitfalls.md` (멀티에이전트), `zen-ai-state-machines.md` (상태 머신 대입)

실습 코드 — `03-graph/toy/` · `python test_mini_minutes.py`

원문 출처 — Anthropic *Building effective agents* / *How we built our multi-agent research system* · Cognition *Don't Build Multi-Agents* · LangChain *How and when to build multi-agent systems* · LangGraph Graph API · Persistence · Interrupts · Temporal / Inngest durable execution